Jiashuo Wang(364341)- DBSAssignment5

Task A

The current toy_store table structure will cause severe data anomalies because it mixes multiple entities (Store, Sale, Item, Clerk) into a single flat table. Here are concrete examples:

1. Insert Anomaly

Description: We cannot insert information about a new entity until it is involved in a transaction.

Concrete Example: If PlayDate group opens a new toy store called "KidsWorld" at "New Street 1", we cannot store this information in the database until they make their first sale. This is because every row in this table requires a receiptNo (a sale transaction) to exist.

2. Update Anomaly

Description: Modifying a piece of data requires updating multiple rows, which can lead to inconsistencies if not done perfectly.

Concrete Example: The clerk "Peter" (clerkId C11) has made multiple sales (e.g., receiptNo 135 and 266). His name is duplicated across these rows. If Peter changes his name to "Pete", we must find and update every single row where clerkId = C11. If we miss updating row 266, the database will have inconsistent data showing C11 as both "Peter" and "Pete".

3. Delete Anomaly

Description: Deleting a record to remove one piece of information unintentionally deletes other unrelated, valuable information.

Concrete Example: Look at the sale made by clerk "Axel" (clerkId C24) on receiptNo 266 at ToyGalaxy. This is his only recorded sale. If the customer returns the items and we delete this receipt record from the table, we completely lose the information that the PlayDate group employs a clerk named Axel with ID C24.

Task B

storeName	storeAddress	receiptNo	saleTime	clerkId	clerkName	sku	itemName	qty	promoCode	PromoCode	warrantyCode	warrantyMonths
PlayWorld	OtherStreet3	135	2025-08-26 10:12	C11	Peter	101	LEGO	1	PR10	Summer	WR10	10
/												
/												
/												
/												
/												
/												
/												

Task C

PK {storeName, receiptNo, sku}

Full Dependencies:

{storeName, receiptNo, sku} -> qty

{storeName, receiptNo, sku} -> promoCode

{storeName, receiptNo, sku} -> warrantyCode

Partial Dependencies:

{storeName, receiptNo} -> saleTime

{storeName, receiptNo} -> clerkId

sku -> itemName

Transitive Dependencies:

storeName -> storeAddress

clerkId -> clerkName

promoCode -> promoName

warrantyCode -> warrantyMonths

Task D

Candidate Keys:

Based on the 1NF table data, the minimal set of attributes that uniquely identifies a row must include the store, the specific transaction, and the specific item bought in that transaction. Therefore, the candidate keys are:

{storeName, receiptNo, sku}

{storeName, receiptNo, itemName}

Primary Key:

I have chosen {storeName, receiptNo, sku} as the Primary Key.

Reason:

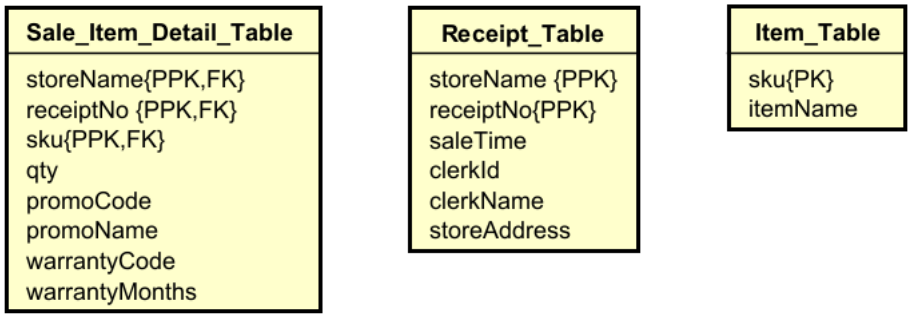
I chose this combination because sku is a numeric/alphanumeric identifier, which is significantly more stable, shorter, and faster to index and query than a descriptive string like itemName. Item names might change over time or contain typos, making {storeName, receiptNo, itemName} an unstable and poorly performing choice for a Primary Key.

Task E

To achieve 2NF, I must eliminate all partial dependencies from the 1NF table. Our primary key is the composite key {storeName, receiptNo, sku}.

1. The attribute itemName only depends on sku, not the full key. I separated this into a new table Item.
2. The attributes related to the transaction itself (saleTime, clerkId, clerkName, storeAddress) only depend on {storeName, receiptNo}, completely ignoring the specific sku being sold. I separated these into a new table Receipt.
3. The remaining attributes (qty, promoCode, promoName, warrantyCode, warrantyMonths) depend on the full composite key and remain in the detail table.

Resulting Tables:



Task F

To achieve 3NF, I need to eliminate all transitive dependencies (where a non-key attribute depends on another non-key attribute).

1. In the Receipt table, clerkName depends on clerkId. I created a new Clerk table.
2. In the Receipt table, storeAddress technically depends only on storeName. To normalize this, I created a new Store table.
3. In the Sale_Item_Detail table, promoName depends on promoCode. I created a new Promotion table.
4. In the Sale_Item_Detail table, warrantyMonths depends on warrantyCode. I created a new Warranty table.

Resulting 3NF Tables: